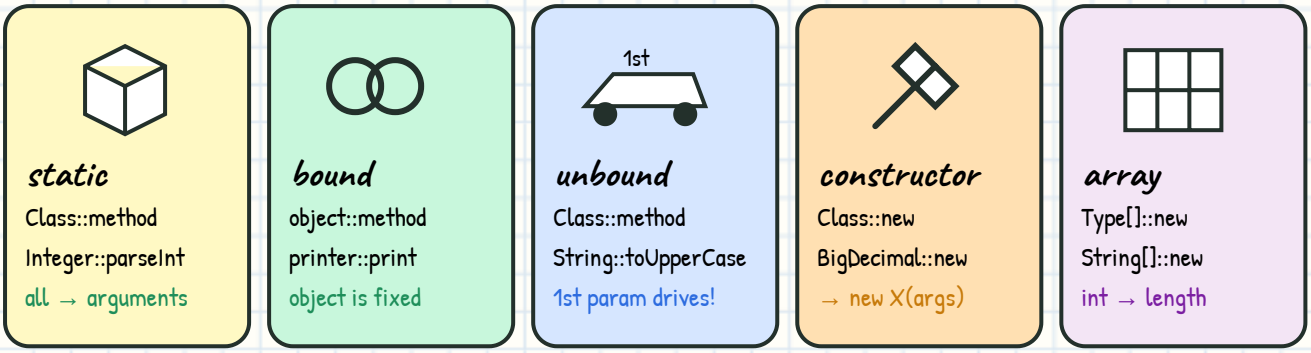
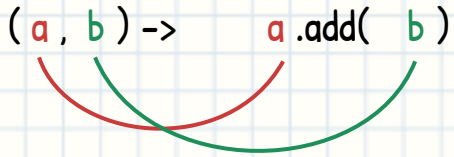
Method references · my notes 04

a lambda that only calls ONE method → just name the method with ::

1 · Four kinds (+ arrays)



2 · Unbound: where do a and b go?



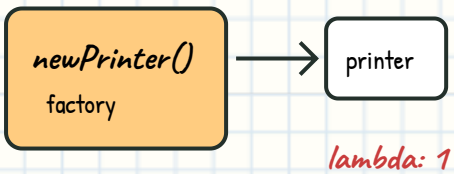
= BigDecimal::add

a = the receiver (the object)
b = the argument

perfect for `reduce(ZERO, BigDecimal::add)`
→ 300382.25

records [16] → free refs:
`Transaction::amount`

3 · Bound receiver = evaluated ONCE (eager)



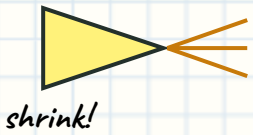
creating both: new | 2 ref calls: | 2 lambda calls: new new

null receiver?

`code::length` with code == null
→ NPE immediately, at creation

`() -> code.length()`
→ NPE only when called

4 · The shrink ray: lambda → reference



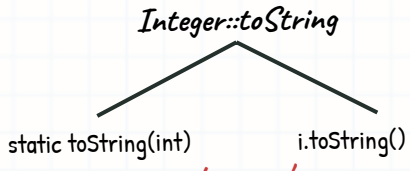
t -> t.amount()	- - - - - - - - - - -	Transaction::amount
s -> new BigDecimal(s)	- - - - - - - - - - -	BigDecimal::new
r -> printer.print(r)	- - - - - - - - - - -	printer::print
n -> new String[n]	- - - - - - - - - - -	String[]::new
o -> Objects.nonNull(o)	- - - - - - - - - - -	Objects::nonNull

5 · Keep the lambda when...

- extra argument: `a -> fee(a, RATE)`
- a chain: `t -> t.amount().compareTo(LIMIT) > 0`
- ambiguous overload
- the lambda is simply clearer

...or give it a name → `::isHighValue`

6 · Integer::toString ?!



ambiguous!

fix: `i -> i.toString()` or `String::valueOf`

7 · 5 parameters?

JDK stops at `BiFunction`. Make my own SAM:

```
CustomerFactory f = Customer::new;
```

Function<String, Customer> → invalid constructor reference

★ Remember

- :: = a lambda that calls ONE method
- unbound: 1st param = receiver
- bound: receiver evaluated once
- good method names make refs shine
- next → L05 Optional